

## Experiment-4

**Aim-** Implement principle component analysis (PCA) technique on 2-D data and plot PCA space of the first two PCs.

### Theory:-

Principal Component Analysis (PCA) is a dimensionality reduction technique used to reduce the number of variables in a dataset while retaining most of the important information. It transforms the original correlated features into a new set of uncorrelated variables called **principal components**.

The first principal component captures the maximum variance in the data, the second captures the next highest variance, and so on.

Steps involved in PCA:

1. Standardize the dataset so all features have the same scale.
2. Compute the covariance matrix of the features.
3. Calculate eigenvalues and eigenvectors.
4. Sort eigenvectors based on eigenvalues.
5. Select the top principal components.
6. Transform the original data into the new feature space.

In this experiment, PCA is applied using the Scikit-learn library to reduce the Iris dataset from **4 dimensions to 2 dimensions** for visualization and analysis.

The explained variance ratio indicates how much information each principal component retains from the original dataset.

## **Algorithm:-**

**Step 1:** Import required libraries such as NumPy, Matplotlib, and PCA from Scikit-learn.

**Step 2:** Load the Iris dataset.

**Step 3:** Separate the feature matrix (X) and target labels (y).

**Step 4:** Standardize the feature values so all variables have the same scale.

**Step 5:** Initialize PCA with number of components = 2.

**Step 6:** Fit PCA on the standardized dataset and transform the data to obtain the reduced dataset.

**Step 7:** Calculate and display:

- Variance explained by each principal component
- Total variance retained
- Information lost after reduction.

**Step 8:** Plot a **2D scatter plot** using:

- PC1 on x-axis
- PC2 on y-axis
- Different colors for each species.

**Step 9:** Extract the principal component vectors.

**Step 10:** Plot a **heatmap** showing contribution of each original feature to each principal component.

## Code:-

```
11 import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
11 # Load dataset
iris = load_iris()

X = iris.data      # Features
y = iris.target    # Target labels
feature_names = iris.feature_names
target_names = iris.target_names

# Standardize features (mean=0, variance=1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Shape of original data:", X.shape)
```

... Shape of original data: (150, 4)

```
11 pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

explained_variance = pca.explained_variance_ratio_

print("Explained variance by PC1:", explained_variance[0])
print("Explained variance by PC2:", explained_variance[1])

total_retained = np.sum(explained_variance)
information_lost = 1 - total_retained

print("\nTotal Information Retained:", total_retained)
print("Total Information Lost:", information_lost)
```

Explained variance by PC1: 0.7296244541329991  
Explained variance by PC2: 0.22850761786701745

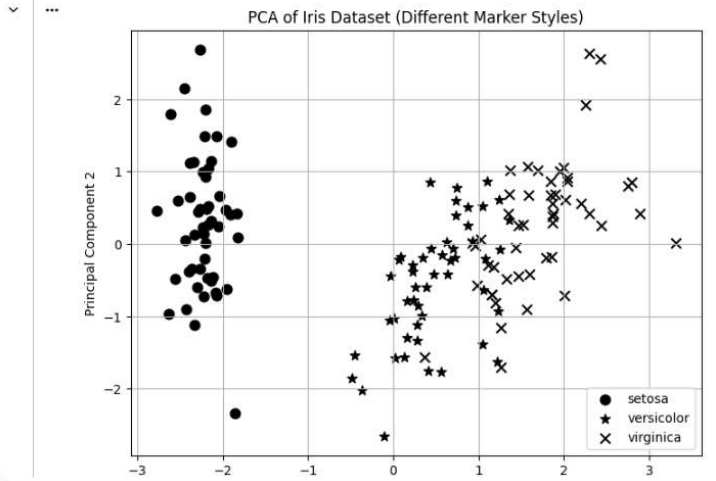
Total Information Retained: 0.9581320720000166  
Total Information Lost: 0.0418679279999834

```
[1] plt.figure(figsize=(8,6))

markers = ['o', '*', 'x'] # circle, square, triangle

for i, target_name in enumerate(target_names):
    plt.scatter(X_pca[y == i, 0],
                X_pca[y == i, 1],
                marker=markers[i],
                color='black',
                label=target_name,
                s=60)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA of Iris Dataset (Different Marker Styles)")
plt.legend()
plt.grid()
plt.show()
```



```
[1] # PCA component vectors (loadings)
loadings = pca.components_

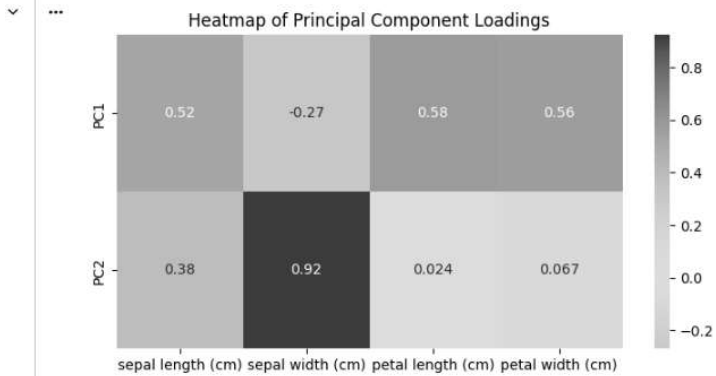
# Convert to DataFrame for better visualization
loadings_df = pd.DataFrame(loadings,
                            columns=feature_names,
                            index=["PC1", "PC2"])

print(loadings_df)
```

|     | sepal length (cm) | sepal width (cm) | petal length (cm) | petal width (cm) |
|-----|-------------------|------------------|-------------------|------------------|
| PC1 | 0.521066          | -0.269347        | 0.580413          | 0.564857         |
| PC2 | 0.377418          | 0.923296         | 0.024492          | 0.066942         |

```
[1] plt.figure(figsize=(8,4))
sns.heatmap(loadings_df,
            annot=True,
            cmap="coolwarm",
            center=0)

plt.title("Heatmap of Principal Component Loadings")
plt.show()
```



## Results and Discussions

PCA successfully reduced the Iris dataset from **4 dimensions to 2 dimensions** while retaining most of the important information. The first two principal components captured the majority of the variance in the dataset. The 2D scatter plot clearly separated the three species, and the heatmap illustrated how strongly each feature contributed to the principal components.